

CI and CD Pipeline Setup

Project Overview

This project implements an end-to-end Continuous Integration and Continuous Deployment (CI/CD) pipeline that automatically builds, packages, and deploys an application whenever changes are pushed to GitHub.

The solution integrates:

- GitHub
- Jenkins (Declarative Pipeline written in Groovy)
- Docker
- AWS Elastic Container Registry (ECR)
- Kubernetes (kubeadm)
- Prometheus
- Grafana
- Loki

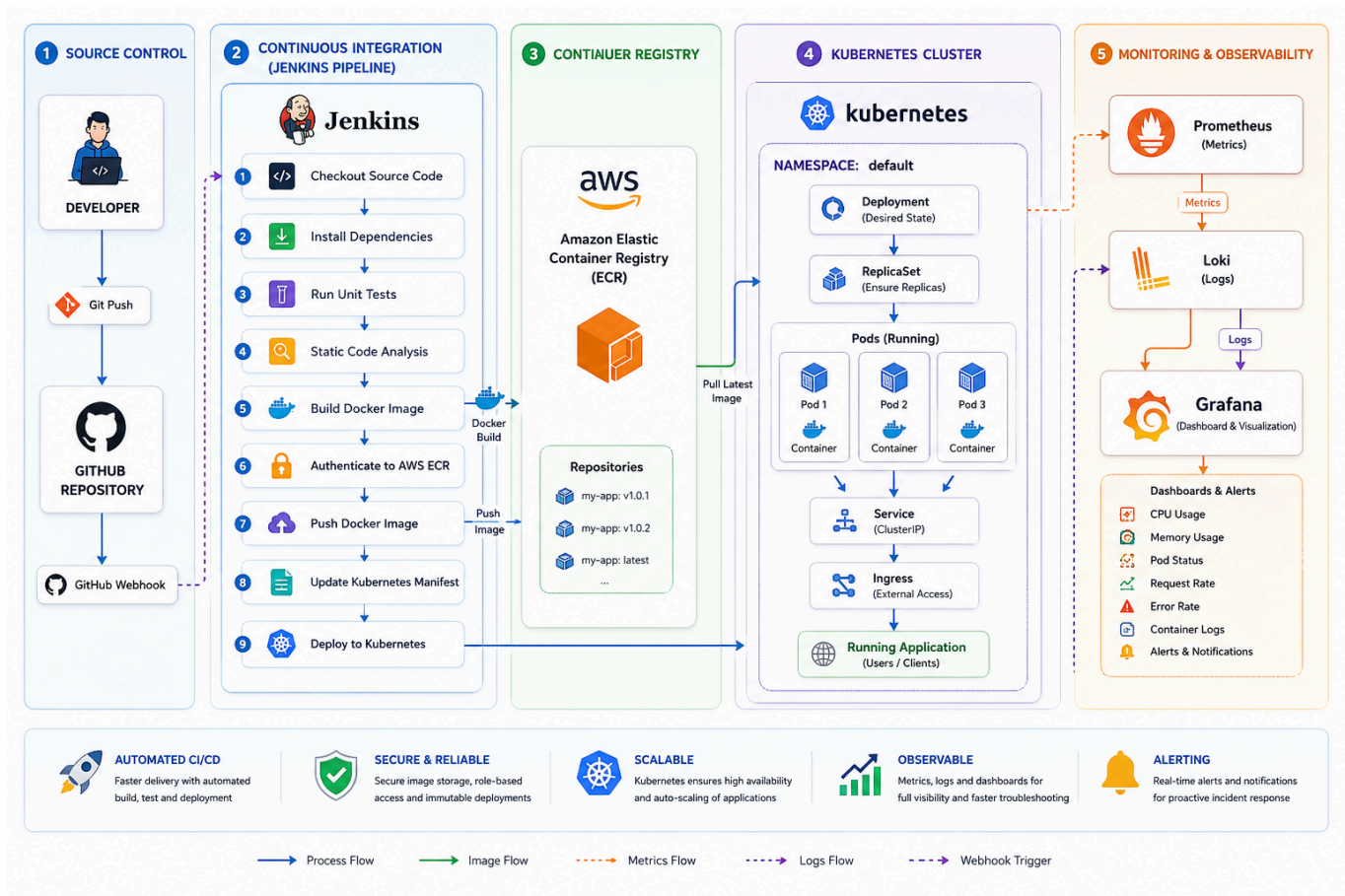
The pipeline minimizes manual intervention, ensures consistent deployments, and provides centralized monitoring and logging.

Problem Statement

Build a scalable CI/CD pipeline that:

- Automatically triggers builds when code is pushed to GitHub.
 - Builds Docker images using Jenkins.
 - Pushes images to AWS Elastic Container Registry (ECR).
 - Deploys applications automatically to Kubernetes.
 - Uses Groovy (Jenkins Pipeline) for automation.
 - Provides monitoring and centralized logging.
 - Follows security best practices.
-

Architecture



Technology Stack

Component	Technology
Version Control	GitHub
CI/CD	Jenkins
Pipeline	Declarative Pipeline (Groovy)
Containerization	Docker
Registry	AWS ECR
Orchestration	Kubernetes
Monitoring	Prometheus
Visualization	Grafana
Logging	Loki

Operating System & Infrastructure

The project uses two Virtual Machines:

VM-1 : Jenkins Server

Installed Components:

- Jenkins
- Docker
- AWS CLI
- kubectl
- Git

Responsibilities:

- Clone Git repository
- Build Docker image
- Push image to AWS ECR
- Deploy application to Kubernetes

VM-2 : Kubernetes Cluster

Installed Components:

- kubeadm
- kubelet
- kubectl
- Container Runtime
- Calico Network Plugin

Responsibilities:

- Run application containers
- Expose services
- Perform rolling updates
- Health monitoring

Repository Structure

```
cicd-jenkins-k8s/
```

```
|
```

```
├─ cicdapp/
|   ├─ Dockerfile
|   ├─ package.json
|   ├─ index.js
|   └─ package-lock.json
|
├─ k8s/
|   ├─ deployment.yaml
|   └─ service.yaml
|
├─ Jenkinsfile
|
└─ README.md
```

CI/CD Workflow

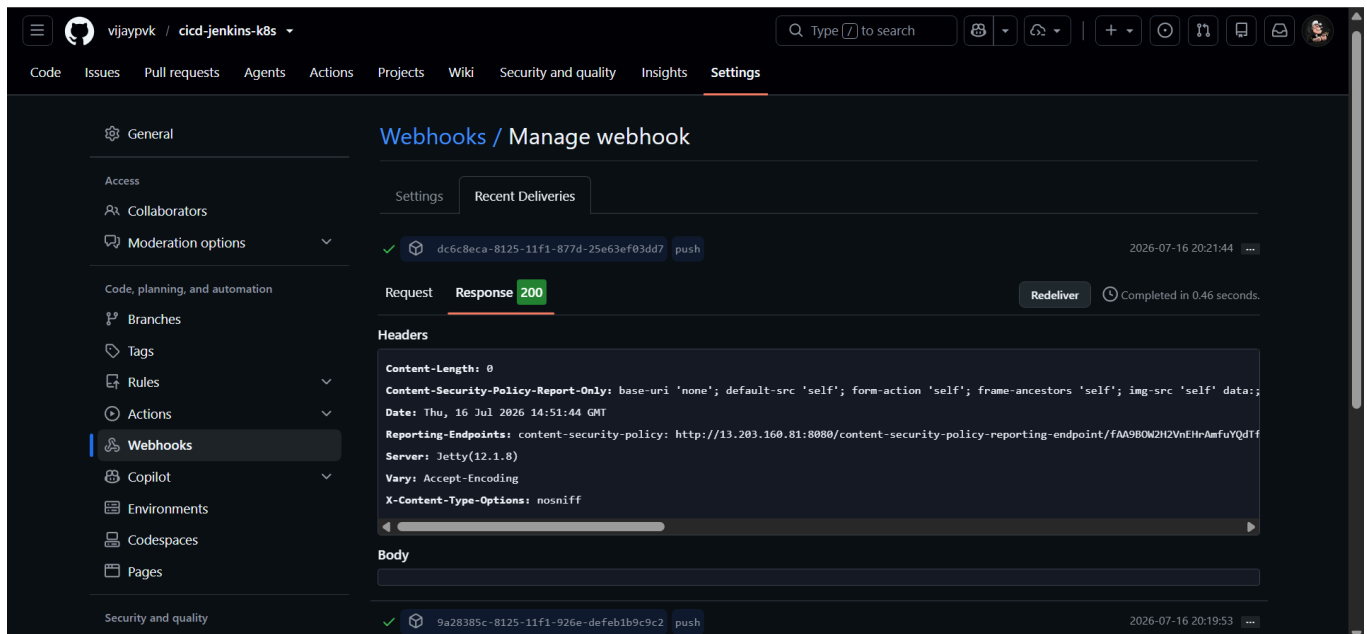
- **Step 1:** Developer pushes code to GitHub.

```
git add .

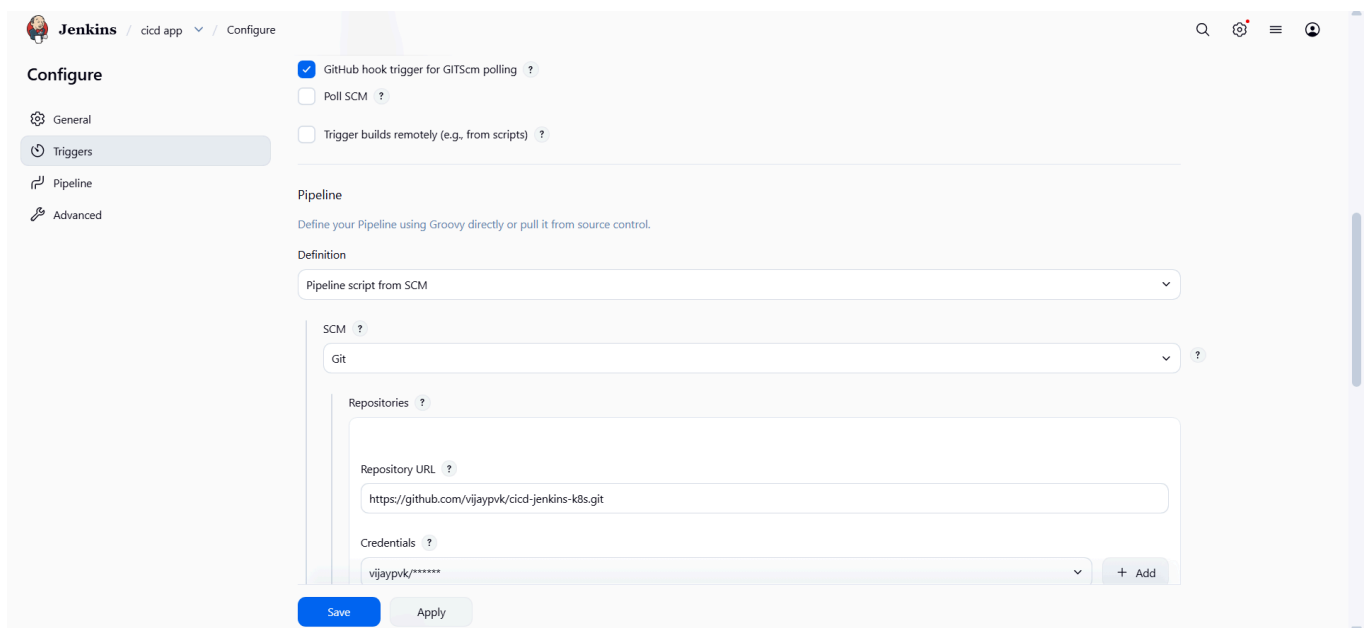
git commit -m "Feature Update"

git push origin main
```

- **Step 2:** GitHub Webhook automatically notifies Jenkins. No manual build is required.



- **Step 3:** Jenkins Pipeline starts automatically. Pipeline stages include:
 - Checkout
 - Build Docker Image
 - Login to AWS ECR
 - Push Image
 - Deploy to Kubernetes
 - Verify Deployment



- **Step 4:** Docker image is built.

```

    \ \ \

    docker build -t cicdapp:latest .

    \ \ \

```

- **Step 5:** Image is pushed to AWS ECR.

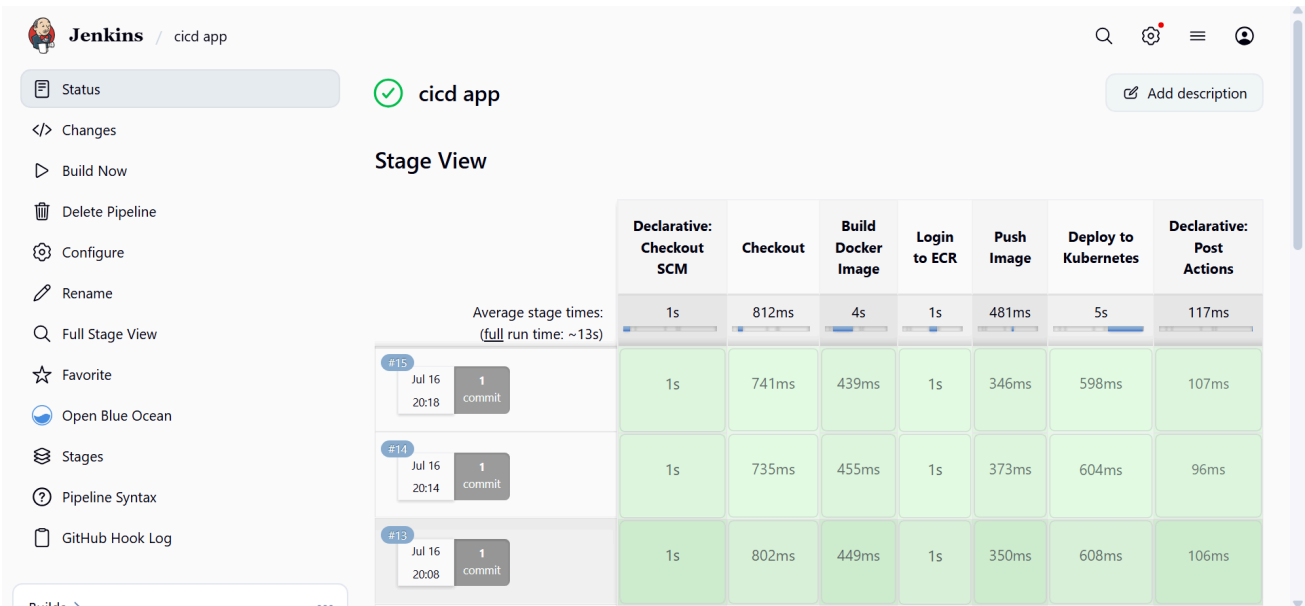
```

    \ \ \

    docker push 775412354718.dkr.ecr.ap-south-1.amazonaws.com/cicdapp:latest

    \ \ \

```



- **Step 6:** Jenkins deploys the latest image to Kubernetes.

```

    \ \ \

    kubectl set image deployment/cicdapp cicdapp=775412354718.dkr.ecr.ap-
    south-1.amazonaws.com/cicdapp:latest

    \ \ \

```

- **Step 7:** Kubernetes performs a rolling update:

```

    \ \ \

    Old Pods —> New Pods —> Health Checks —> Traffic Switch —> Deployment
    Complete

    \ \ \

```

No downtime occurs during deployment.


```

}

stage('Build Docker Image') {

    steps {

        dir('cicdapp') {

            sh '''
            docker build -t ${IMAGE} .
            '''

        }

    }

}

stage('Login to ECR') {

    steps {

        withCredentials([
            [$class: 'AmazonWebServicesCredentialsBinding',
             credentialsId: 'aws-creds']
        ]) {

            sh '''
            aws ecr get-login-password --region ${AWS_REGION} \
            | docker login --username AWS --password-stdin
            ${ECR_REGISTRY}
            '''

        }

    }

}

stage('Push Image') {

    steps {

        sh '''
        docker push ${IMAGE}
        '''

    }

}

```



```

    }

}

stage('Deploy to Kubernetes') {
    steps {
        sh '''
            export KUBECONFIG=/var/lib/jenkins/.kube/config

            kubectl set image deployment/cicdapp \
            cicdapp=${IMAGE}

            kubectl rollout status deployment/cicdapp
        '''
    }
}

}

post {

    success {

        echo "Deployment Successful"

    }

    failure {

        echo "Deployment Failed"

    }

}

}

```

The CI/CD automation is implemented using a **Jenkins Declarative Pipeline**.

- **Pipeline stages:** Checkout Source Code, Build Docker Image, Login to AWS ECR, Push Docker Image, Deploy to Kubernetes, Verify Rollout.
- **Post Build Actions:** Handles success and failure blocks for cleanups and notifications.
- Since Jenkinsfiles are written in Groovy, the assignment requirement for Groovy scripting is fully satisfied.

Jenkins Plugins Used

- Git Plugin
 - Docker Plugin
 - Docker Pipeline
 - Kubernetes CLI Plugin
 - Credentials Plugin
 - AWS Credentials Plugin
 - Pipeline Plugin
 - Blue Ocean (Optional)
-

Kubernetes Deployment & AWS ECR Integration

Application Architecture

The application is deployed using native Kubernetes manifests:

- **Deployment:** Configured with specific Replica Counts, Rolling Update Strategy, Liveness Probes, and Readiness Probes.

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: cicdapp

spec:
  replicas: 2

  selector:
    matchLabels:
      app: cicdapp

  template:
    metadata:
      labels:
        app: cicdapp

    spec:
      imagePullSecrets:
        - name: ecr-secret
```

```

containers:
- name: cicdapp
  image: 775412354718.dkr.ecr.ap-south-1.amazonaws.com/cicdapp:latest
  imagePullPolicy: Always

ports:
- containerPort: 3000

readinessProbe:
  httpGet:
    path: /
    port: 3000
  initialDelaySeconds: 10
  periodSeconds: 5

livenessProbe:
  httpGet:
    path: /
    port: 3000
  initialDelaySeconds: 20
  periodSeconds: 10

```

- **Service:** NodePort Service maps external traffic to pods.

```

apiVersion: v1
kind: Service

metadata:
  name: cicdapp-service

spec:
  selector:
    app: cicdapp

  ports:
    - protocol: TCP
      port: 80
      targetPort: 3000

  type: NodePort

```

Readiness Probe —> Checks application availability

Liveness Probe → Restarts unhealthy containers automatically

Output :

```
ubuntu@ip-172-31-7-100:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
cicdapp-75f4ddcd4b-gd8vg           1/1      Running   0            19h
cicdapp-75f4ddcd4b-w9t66           1/1      Running   0            19h
nginx-7854ff8877-rzbjz             1/1      Running   0            23h

ubuntu@ip-172-31-7-100:~$ kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
cicdapp-service     NodePort    10.106.197.42 <none>         80:31039/TCP     21h
kubernetes           ClusterIP   10.96.0.1      <none>         443/TCP          23h
nginx               NodePort    10.111.8.222   <none>         80:31754/TCP     23h

ubuntu@ip-172-31-7-100:~$ curl http://localhost:31039
CI/CD Pipeline with Git, Jenkins, Docker, AWS ECR & Kubernetes 🚀
```

AWS ECR Integration

Docker images are stored securely inside AWS Elastic Container Registry.

- **Benefits:** Central Image Repository, Version Control, Secure Infrastructure, and Easy Rollback capabilities.

Monitoring & Logging Strategy

The project includes a centralized monitoring and logging stack consisting of **Prometheus**, **Loki**, and **Grafana**.

Monitoring (Prometheus)

Monitoring is implemented using Prometheus and Grafana.

Metrics monitored include:

- Jenkins build success/failure
- Pipeline execution duration
- Jenkins queue
- Executor availability
- CPU utilization

- Memory utilization
- JVM metrics
- Kubernetes node health
- Pod health
- Deployment availability
- Cluster resource usage

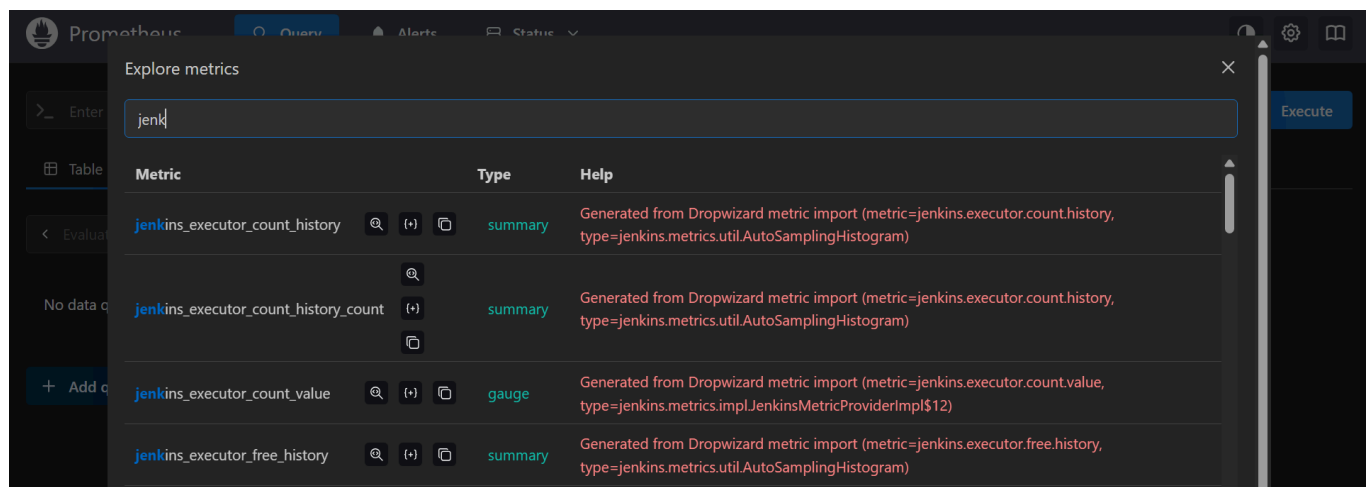
Benefits

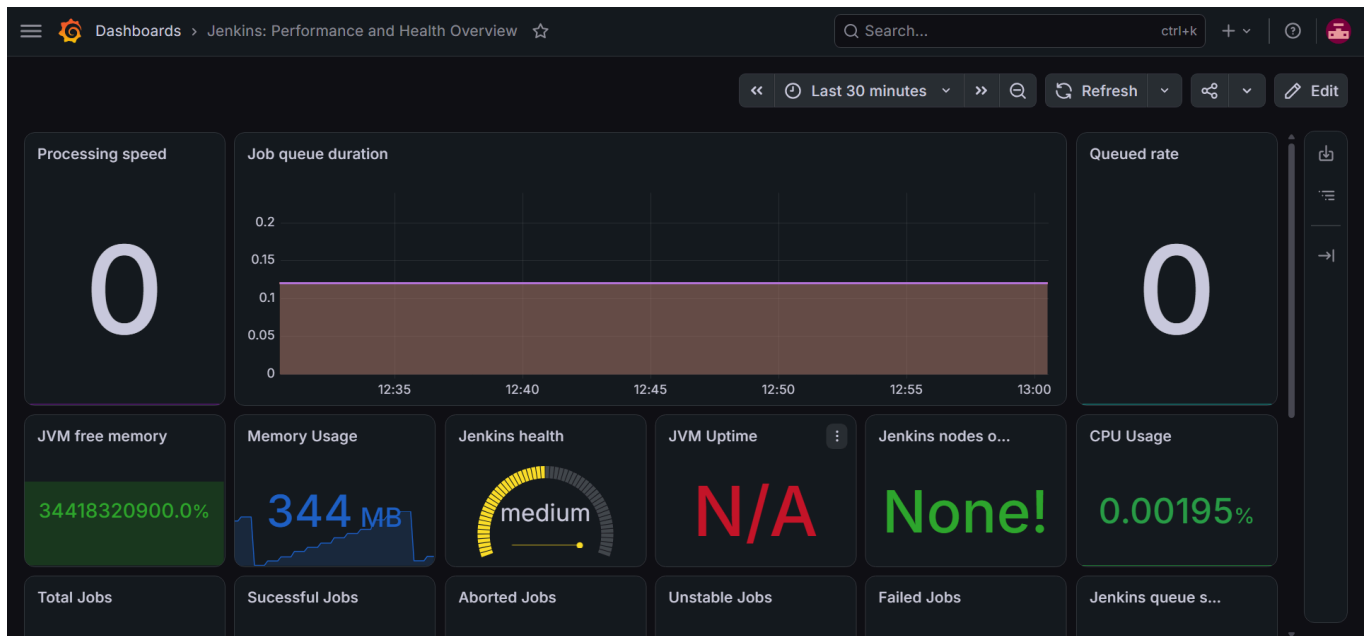
- Detect pipeline failures
- Detect infrastructure bottlenecks
- Monitor deployment health
- Capacity planning

output:

```
ubuntu@ip-172-31-7-100:~$ kubectl get svc -n monitoring
NAME                                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
alertmanager-operated              ClusterIP    None             <none>            9093/TCP,9094/TCP,9094/UDP            9h
loki                                ClusterIP    10.111.87.176    <none>            3100/TCP                              7h48m
loki-headless                      ClusterIP    None             <none>            3100/TCP                              7h48m
loki-memberlist                   ClusterIP    None             <none>            7946/TCP                              7h48m
monitoring-grafana                 NodePort     10.101.14.26     <none>            80:30412/TCP                          9h
monitoring-kube-prometheus-alertmanager ClusterIP    10.99.249.192    <none>            9093/TCP,8080/TCP                    9h
monitoring-kube-prometheus-operator ClusterIP    10.98.131.76     <none>            443/TCP                              9h
monitoring-kube-prometheus-prometheus NodePort     10.97.88.71      <none>            9090:32142/TCP,8080:31219/TCP        9h
monitoring-kube-state-metrics      ClusterIP    10.102.219.37    <none>            8080/TCP                              9h
monitoring-prometheus-node-exporter ClusterIP    10.99.33.95      <none>            9100/TCP                              9h
prometheus-operated                ClusterIP    None             <none>            9090/TCP                              9h
ubuntu@ip-172-31-7-100:~$ |

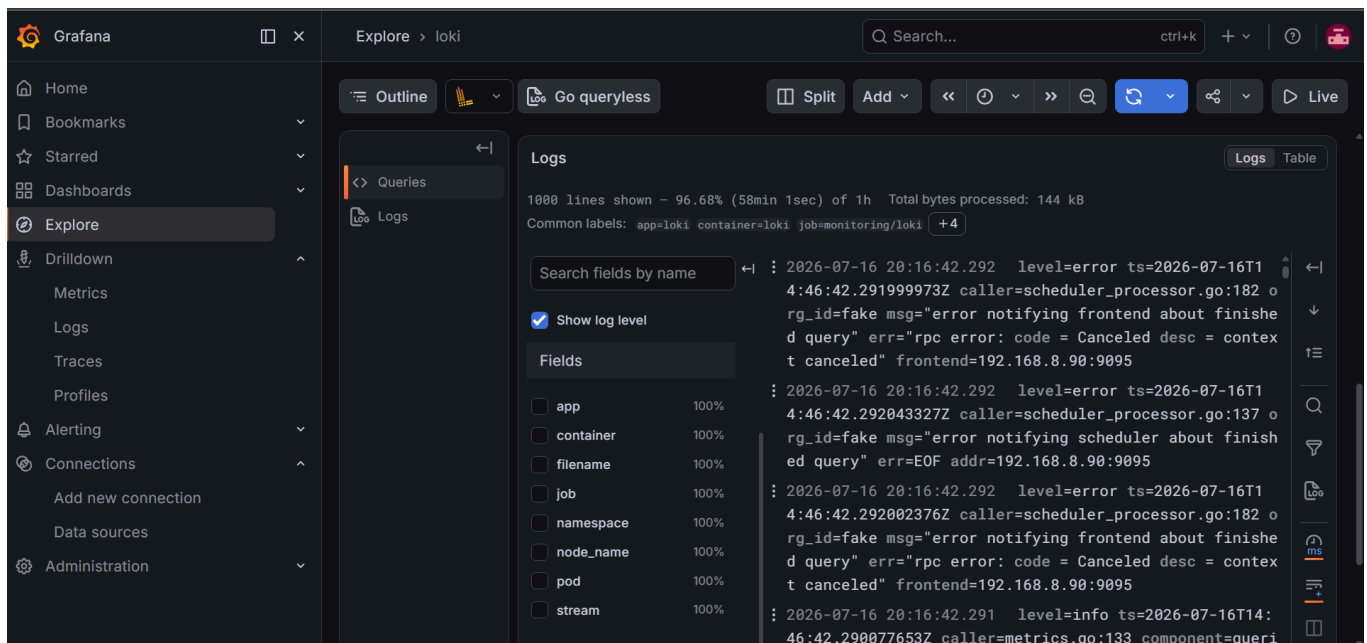
ubuntu@ip-172-31-7-100:~$ helm list -n monitoring
NAME      NAMESPACE    REVISION    UPDATED                               STATUS    CHART              APP VERSION
loki      monitoring    1           2026-07-16 06:52:28.490445935 +0000 UTC deployed  loki-stack-2.10.3  v2.9.3
monitoring monitoring    2           2026-07-16 05:33:11.421237644 +0000 UTC deployed  kube-prometheus-stack-87.16.1 v0.92.1
ubuntu@ip-172-31-7-100:~$ |
```





Logging (Loki)

- **Logs Collected:** Kubernetes Pod Logs, Application Logs, Jenkins Logs, and Container Logs.
- **Benefits:** Faster Troubleshooting, Centralized Log Storage, Searchable Logs, and Historical Log Analysis.



Visualizations (Grafana Dashboards)

- **Jenkins Dashboard:** Successful/Failed Builds, Queue Length, Build Duration, Jenkins Health, JVM Memory, CPU Usage, and Executor Status.
- **Kubernetes Dashboard:** Cluster Health, Pod Status, Deployment Status, CPU/Memory Usage, and Node Utilization.



Security, Scalability, & Error Handling

Security Best Practices

- **Jenkins Credentials:** AWS Access Keys are stored securely inside Jenkins Credentials; passwords/keys are never hardcoded inside the Jenkinsfile.
- **Kubernetes Secrets:** Docker registry authentication is handled via `imagePullSecrets`.
- **Least Privilege:** Only required AWS IAM permissions are granted to the execution role.
- **Secure Communication:** Jenkins communicates securely with Kubernetes using encrypted `kubeconfig`.

Error Handling & Validation

The pipeline performs automatic validation post-deployment:

```
kubectl rollout status deployment ciccapp
```

If the deployment fails, the pipeline stops, the error logs are caught, and the status is marked as **Failed** to prevent shipping broken changes.

Scalability

The solution is fully parameterized and reusable. Changing the configuration variables enables the same engine to support multiple repositories, namespaces, images, and Kubernetes

clusters.

Validation & Testing

Deployment validation

```
kubectl get pods  
kubectl get deployment  
kubectl get svc
```

Application validation

```
curl http://<NODE-IP>:<NODEPORT>
```

Expected Output

CI/CD Pipeline with Git, Jenkins, Docker, AWS ECR & Kubernetes 🚀

Monitoring validation

- Jenkins dashboard available
- Kubernetes dashboard available
- Prometheus targets healthy

Logging validation

- Logs searchable in Grafana Explore
-

Conclusion

This project successfully satisfies the assignment requirements by implementing an automated CI/CD pipeline using GitHub, Jenkins (Groovy), Docker, AWS ECR, and Kubernetes.

The pipeline automatically builds and deploys applications after every code commit, reducing manual intervention and ensuring consistent software delivery.

Security best practices, deployment validation, centralized monitoring with Prometheus and Grafana, and centralized logging with Loki have also been implemented, resulting in a robust, scalable, and production-oriented CI/CD solution.